

Laboratorium 4: Kodowanie znaków w komputerze

Opracowanie zestawu zadań (część teoretyczna)

Kacper Bednarczuk

Zadanie 1.1

Tekst `Kod: 007!` po zakodowaniu w 7-bitowym standardzie ASCII przyjmuje postać: `4B 6F 64 3A 20 30 30 37 21` (reprezentacja heksadecymalna).

Poszczególne znaki zostały zakodowane w następujący sposób:

- `K` – `4B`
 - `o` – `6F`
 - `d` – `64`
 - `:` – `3A`
 - – `20`
 - `0` – `30`
 - `0` – `30`
 - `7` – `37`
 - `!` – `21`
-

Zadanie 1.2

1. Problemy z reprezentacją w 7-bitowym ASCII

Próba zakodowania zdania `Zażółć gęślą jaźń` w 7-bitowym standardzie ASCII jest niemożliwa, ponieważ następujące znaki diakrytyczne nie mają w nim swoich odpowiedników: `ż`, `ó`, `ł`, `ć`, `ę`, `ś`, `ą`, `ź`, `ń`.

Jest tak, ponieważ standard 7-bitowy ASCII został zaprojektowany dla języka angielskiego i obejmuje tylko 128 znaków, w tym litery bez znaków diakrytycznych, cyfry i podstawowe znaki interpunkcyjne. Nie przewidziano w nim miejsca na znaki specyficzne dla innych języków.

2. Wynik kodowania z pominięciem znaków diakrytycznych

Gdyby program zignorował znaki diakrytyczne, zamieniając je na ich podstawowe odpowiedniki (np. $\acute{z} \rightarrow z$), oryginalne zdanie przyjąłoby postać: Zazolc gesla jazn .

Zakodowanie takiego uproszczonego tekstu w standardzie ASCII dałoby następujący ciąg kodów szesnastkowych:

5A 61 7A 6F 6C 63 20 67 65 73 6C 61 20 6A 61 7A 6E

Poszczególne znaki zostały zakodowane w następujący sposób:

- Z – 5A
- a – 61
- z – 7A
- o – 6F
- ł – 6C
- c – 63
- – 20
- g – 67
- e – 65
- s – 73
- ł – 6C
- a – 61
- – 20
- j – 6A
- a – 61
- z – 7A
- n – 6E

Zadanie 1.3

Zgodnie z kodami podanymi w poleceniu, słowo Zażółć w standardzie **ISO-8859-2** przyjmuje postać:

5A 61 BF F3 B3 E6

Note

Rozwiązanie polega na bezpośrednim wykorzystaniu kodów szesnastkowych przypisanych do poszczególnych znaków w treści zadania.

Zadanie 1.4

Kodowanie słowa **Zażółć** w standardzie UTF-8 wymaga przetworzenia każdego znaku zgodnie z jego numerem Unicode (codepoint).

Krok 1: Zakodowanie litery **z**

- **Codepoint:** U+005A
- **Analiza:** Numer 0x5A (dziesiętnie 90) mieści się w zakresie U+0000 – U+007F . Oznacza to, że znak jest kodowany na **jednym bajcie**, identycznie jak w ASCII.
- **Szablon:** 0xxxxxxx
- **Binarnie:** 0x5A = 1011010
- **Wynik:** 01011010 (binarnie) = 5A (heksadecymalnie)

Krok 2: Zakodowanie litery **a**

- **Codepoint:** U+0061
- **Analiza:** Numer 0x61 (dziesiętnie 97) również mieści się w zakresie U+0000 – U+007F . Znak jest kodowany na **jednym bajcie**.
- **Szablon:** 0xxxxxxx
- **Binarnie:** 0x61 = 1100001
- **Wynik:** 01100001 (binarnie) = 61 (heksadecymalnie)

Krok 3: Zakodowanie litery **ż**

- **Codepoint:** U+017C
- **Analiza:** Numer 0x017C (dziesiętnie 380) mieści się w zakresie U+0080 – U+07FF . Oznacza to, że znak musi zostać zakodowany na **dwóch bajtach**.
- **Szablon:** 110xxxxx 10xxxxxx
- **Binarnie:** 0x017C = 0001 0111 1100 . Potrzebujemy 11 bitów do wypełnienia szablonu: 00101 111100 .
 1. Sześć ostatnich bitów (111100) trafia do drugiego bajtu: 10**111100** .
 2. Pozostałe pięć bitów (00101) trafia do pierwszego bajtu: 110**00101** .
- **Wynik:** 11000101 10111100 (binarnie) = C5 BC (heksadecymalnie)

Krok 4: Zakodowanie litery **ó**

- **Codepoint:** U+00F3
- **Analiza:** Numer 0x00F3 (dziesiętnie 243) mieści się w zakresie U+0080 – U+07FF . Znak kodowany na **dwóch bajtach**.
- **Szablon:** 110xxxxx 10xxxxxx
- **Binarnie:** 0x00F3 = 0000 1111 0011 . Potrzebne 11 bitów: 00011 110011 .
 1. Sześć ostatnich bitów (110011) trafia do drugiego bajtu: 10**110011** .

2. Pozostałe pięć bitów (00011) trafia do pierwszego bajtu: 110**00011** .

- **Wynik:** 11000011 10110011 (binarnie) = C3 B3 (heksadecymalnie)

Krok 5: Zakodowanie litery ł

- **Codepoint:** U+0142
- **Analiza:** Numer 0x0142 (dziesiętnie 322) mieści się w zakresie U+0080 – U+07FF .
Znak kodowany na **dwóch bajtach**.
- **Szablon:** 110xxxxx 10xxxxxx
- **Binarnie:** 0x0142 = 0001 0100 0010 . Potrzebne 11 bitów: 00101 000010 .
 1. Sześć ostatnich bitów (000010) trafia do drugiego bajtu: 10**000010** .
 2. Pozostałe pięć bitów (00101) trafia do pierwszego bajtu: 110**00101** .
- **Wynik:** 11000101 10000010 (binarnie) = C5 82 (heksadecymalnie)

Krok 6: Zakodowanie litery ć

- **Codepoint:** U+0107
- **Analiza:** Numer 0x0107 (dziesiętnie 263) mieści się w zakresie U+0080 – U+07FF .
Znak kodowany na **dwóch bajtach**.
- **Szablon:** 110xxxxx 10xxxxxx
- **Binarnie:** 0x0107 = 0001 0000 0111 . Potrzebne 11 bitów: 00100 000111 .
 1. Sześć ostatnich bitów (000111) trafia do drugiego bajtu: 10**000111** .
 2. Pozostałe pięć bitów (00100) trafia do pierwszego bajtu: 110**00100** .
- **Wynik:** 11000100 10000111 (binarnie) = C4 87 (heksadecymalnie)

Podsumowanie

Słowo Założć zakodowane w UTF-8 to następujący ciąg bajtów (heksadecymalnie):

5A 61 C5 BC C3 B3 C5 82 C4 87

Zadanie 1.5

1. Dekodowanie ciągu ASCII

Ciąg bajtów 50 6F 7A 64 72 6F 77 69 65 6E 69 61 20 7A 20 77 79 6B 6C 61 64 75 21
zdekodowany przy użyciu standardu **ASCII** daje tekst: Pozdrowienia z wykładu! .

Poszczególne znaki odpowiadają następującym kodom heksadecymalnym:

- P – 50
- o – 6F
- z – 7A
- d – 64

- r – 72
- o – 6F
- w – 77
- i – 69
- e – 65
- n – 6E
- i – 69
- a – 61
- – 20
- z – 7A
- – 20
- w – 77
- y – 79
- k – 6B
- ł – 6C
- a – 61
- d – 64
- u – 75
- ! – 21

2. Identyfikacja kodowania dla „Cześć!”

Ciąg bajtów 43 7A 65 9C E6 21 odpowiada tekstowi Cześć! . Aby zidentyfikować wykorzystane kodowanie, możemy przeanalizować podane opcje:

- **ASCII:** Odpada, ponieważ nie zawiera polskich znaków diakrytycznych.
- **UTF-8:** Kodowanie ść w UTF-8 to C5 9B C4 87 , co nie pasuje do sekwencji 9C E6 .
- **ISO-8859-2:** Podpowiedź w zadaniu sugeruje, że ść w tym kodowaniu to 9C E6 . Jest to jednak informacja **błędna** (w rzeczywistości 9C to kod kontrolny), kod ten odpowiada natomiast literze ś w kodowaniu **Windows-1250**, które wskazują jako prawidłową odpowiedź.

Wyjaśnienie nieścisłości

W rzeczywistości sekwencja 9C E6 dla znaków ść jest charakterystyczna dla kodowania **Windows-1250**, a nie ISO-8859-2. W standardzie ISO-8859-2 znak ś ma kod B6 , a 9C jest niedrukowalnym kodem kontrolnym. Zakładam, że w zadaniu doszło do pomyłki między tymi dwoma bardzo podobnymi standardami.

3. Skutek błędnej interpretacji (Mojibake)

Gdybyśmy otworzyli plik o zawartości 43 7A 65 9C E6 21 , błędnie interpretując go jako **ISO-8859-2** (tym razem zgodnie z **faktycznym** standardem, a nie błędną podpowiedzią), zobaczylibyśmy:

- 43 → C
- 7A → z
- 65 → e
- 9C → [znak niedrukowalny/krzaczek] (ponieważ 9C to kod kontrolny)
- E6 → ć
- 21 → !

W rezultacie na ekranie pojawiłby się tekst w postaci Cześć! z błędem w miejscu litery ś , np. Cze?ć! lub Cze ć! – jest to klasyczny przykład zjawiska **Mojibake**.

Etymologia słowa Mojibake

Japońskie słowo 文字化け (**Mojibake**), opisujące "krzaczkę" na ekranie, jest bardzo trafne. Składa się z dwóch części:

- 文字 (**moji**) – oznacza "znak" lub "literę".
- 化け (**bake**) – pochodzi od czasownika *bakeru* (化ける), oznaczającego "zmienić się" lub "przetransformować", często w nieoczekiwany lub upiorny sposób.

Dosłownie można to przetłumaczyć jako "upiorna transformacja znaków".